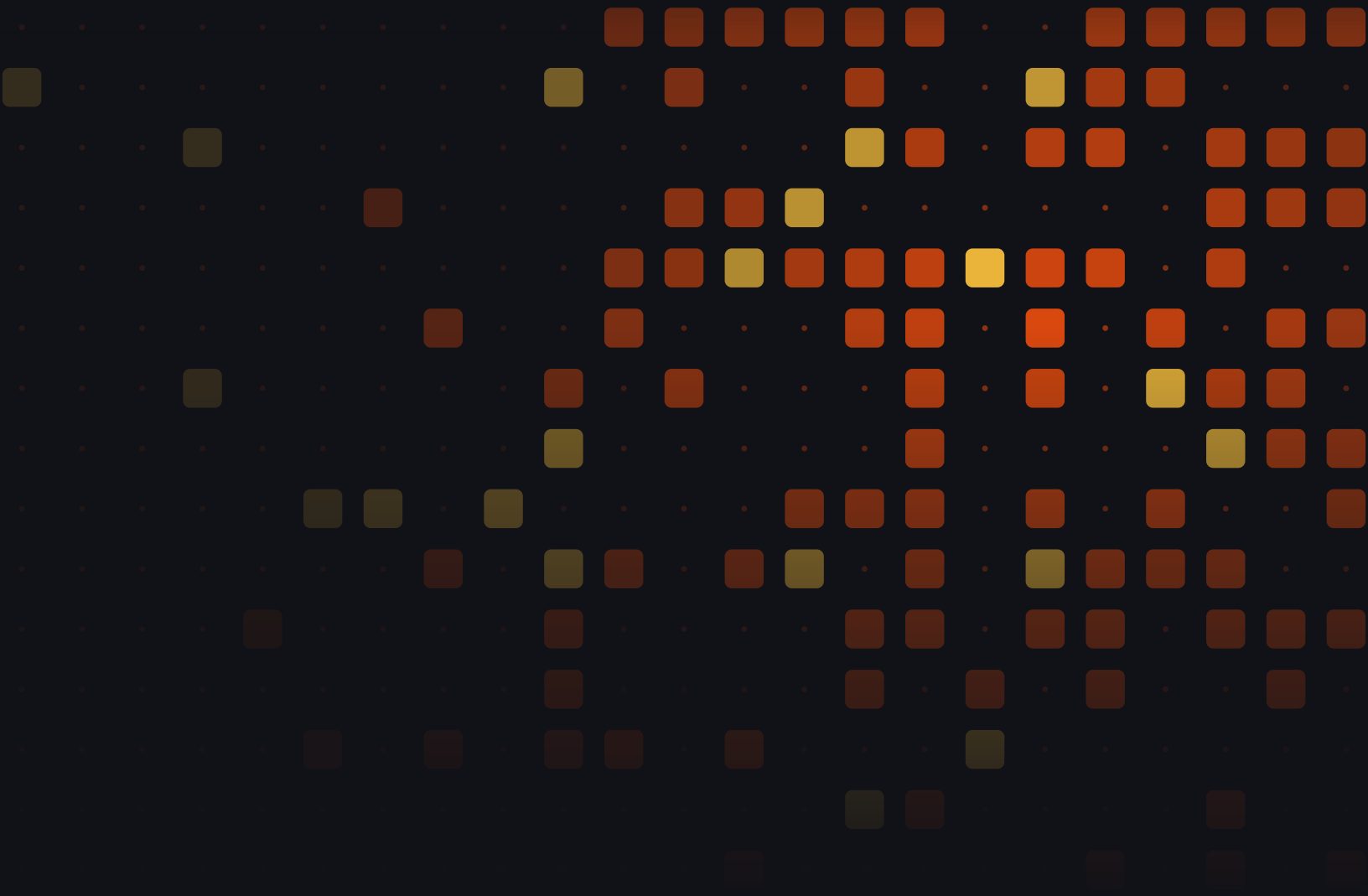


A POWER-USER'S GUIDE

Maximum Autonomy

Every setting, flag, config file, and token choice for hands-off AI agents, tool by tool

For people ready to let agents run • October 2026 edition • Print edition · US Letter

GUIDES FOR EVERYTHING

Maximum Autonomy

Every setting, flag, config file, and token choice for hands-off AI agents, tool by tool

This guide is for one goal: letting your AI agents work **hands-off**, with as few approval prompts as possible, across Claude Code, the Claude apps, Cursor, VS Code and Copilot, Codex, Gemini CLI, GitHub, MCP servers, n8n, and Notion.

You've decided you accept the risk, so this guide doesn't argue with you. It gives you the exact switches. It also shows the one habit that makes maximum autonomy sustainable: **contain the agent instead of restricting it**. Give it a box where mistakes are cheap, then remove the guardrails inside that box. Each chapter ends with what *still* prompts, no matter what you set.

HOW TO READ THIS MANUAL

Tip — a shortcut or good habit.

Note — useful background.

Warning — easy to get wrong.

Danger — can lose data, money, or access.

`$ command` — text you type, exactly as shown.

Settings › Privacy — a menu path to click.

New words appear in **bold** the first time and are collected in the Glossary at the back.

Settings, flags, and menu paths were checked against each vendor's official documentation in October 2026. These products change monthly, and safety prompts in particular are added and renamed often. If a setting name doesn't match what you see, check the vendor's current docs. You are responsible for what your agents do with the access you grant.

Contents

01 **The Autonomy Playbook** 5

1.1 Four levels of autonomy 5

1.2 Contain, don't restrict 5

1.3 Quick-start: maximum autonomy at a glance 6

02 **Build a Safe Playground** 7

2.1 Option 1: a dev container (recommended for coding agents) 7

2.2 Option 2: a dedicated machine or cloud box 8

2.3 Option 3: a separate user account on your computer 8

2.4 Accounts, tokens, and money 8

03 **Claude Code (Terminal)** 9

3.1 Level 3: auto mode by default 9

3.2 Level 4: bypass permissions 10

3.3 Fully unattended runs 11

3.4 A complete maximum-autonomy settings file 11

3.5 Reduce prompts without bypass 12

3.6 What still prompts in Claude Code, no matter what 13

04 **Claude Desktop, Cowork, and claude.ai** 14

4.1 The Code tab in Claude Desktop 14

4.2 Cowork 15

4.3 Connectors in claude.ai and the Claude apps 15

4.4 Claude in Chrome 15

05 **Cursor** 16

5.1 The editor's agent 16

5.2 The Cursor CLI 17

06 **VS Code and GitHub Copilot** 18

6.1 Copilot agent in VS Code 18

6.2 Copilot CLI 19

6.3 Copilot coding agent (on GitHub) 19

07 **Codex, Gemini CLI, and Builders** 20

7.1 OpenAI Codex 20

7.2 Gemini CLI 20

7.3 Browser app builders 21

08 **GitHub: Tokens and Repository Settings** 22

8.1 Authenticate agents broadly 22

8.2 Repository settings that remove friction 23

8.3 What still prompts on GitHub 24

09 **MCP Servers and Connectors** 25

9.1 Give servers enough power 25

10 **n8n** 26

10.1 Let AI agents build and run your workflows 26

10.2 Self-hosted: unlock everything 26

11 **Notion** 28

11.1 Notion MCP (simplest, full personal access) 28

11.2 An API connection with every capability 28

11.3 Notion AI agents 28

12 **What Always Asks: The Master List** 30

Your Autonomy Setup Checklist 31

Glossary 32

CHAPTER 1

The Autonomy Playbook

IN THIS CHAPTER

- The four levels of agent autonomy
- “Contain, don’t restrict”: the principle behind this guide
- The quick-start setup for each tool

1.1 Four levels of autonomy

Every agent tool offers some version of these four levels:

LEVEL	WHAT HAPPENS	EXAMPLES
1. Ask	The agent asks before most actions	Claude Code *Manual*, Cursor *Allowlist* with an empty list
2. Allowlist	Pre-approved actions run; everything else asks	Allow rules, terminal allowlists
3. Classifier	A second AI reviews each action and blocks only risky ones	Claude Code *Auto*, Cursor *Auto-review*
4. Bypass	Everything runs. No checks except a few hard-coded ones	`--dangerously-skip-permissions`, Cursor *Run Everything*, Codex `--yolo`

Level 3 gets you about 95% of the hands-off feel while still catching disasters. Level 4 is true maximum autonomy. This guide gives you both, so you can choose per tool and per project.

1.2 Contain, don’t restrict

Prompts exist to stop an agent from damaging things that matter. You can get the same protection a better way: put the agent somewhere where **nothing it can reach matters too much**. Then you can safely turn the prompts off.

- **A disposable machine:** a container, a virtual machine, a cloud dev box, or a spare computer.

- **Separate accounts:** a dedicated GitHub account or bot for the agent, a separate Notion workspace or teamspace, a separate n8n instance.
- **Narrow-but-complete tokens:** full power *inside* the agent's projects, nothing outside them.
- **Budgets:** spending caps on every API and cloud account.
- **Backups and Git:** everything committed, everything restorable.

Chapter 2 shows how to set this up in an afternoon.

> ☆ IMPORTANT
>
> The single biggest real-world risk of hands-off agents is **prompt injection**: instructions hidden in a web page, email, issue, or document that the agent reads and obeys. Containment limits the damage it can do. Bypass settings remove the last line of defense against it.

1.3 Quick-start: maximum autonomy at a glance

TOOL	HANDS-OFF WITH SAFETY REVIEW	FULL BYPASS
Claude Code (CLI)	`"defaultMode": "auto"` in `~/.claude/settings.json`	`claude --dangerously-skip-permissions`
Claude Desktop, Code tab	Mode selector, then **Auto**	Settings › Claude Code › **Allow bypass permissions mode**, then **Bypass permissions**
Claude Cowork	Mode selector, then **Automatically approve**	(no separate bypass)
claude.ai connectors	Each tool set to **Always allow**	(same)
Cursor	Approvals & Execution: **Auto-review**	Approvals & Execution: **Run Everything**
VS Code + Copilot	**Assisted permissions** (experimental)	**Allow all** / **Autopilot**, `chat.tools.global.autoApprove`
Copilot CLI	Allow rules	`copilot --yolo` (alias of `--allow-all`)
OpenAI Codex	`--full-auto`	`codex --yolo`, or `approval_policy = "never"` + `sandbox_mode = "danger-full-access"`
Gemini CLI	`--approval-mode=auto_edit`	`gemini --yolo`
GitHub	Bot account + fine-grained token	Broad token + Actions write permissions + auto-merge
n8n	Instance-level MCP + API key	Plus `NODES_EXCLUDE="[]"` on self-hosted
Notion	Notion MCP (OAuth)	Internal connection with all capabilities, shared at the top level

CHAPTER 2

Build a Safe Playground

Fifteen minutes of setup here lets you run everything else in this guide at full throttle.

2.1 Option 1: a dev container (recommended for coding agents)

A **dev container** is a Docker container configured as a complete development environment. Anthropic publishes a reference dev container for Claude Code that runs as a **non-root user** with a network firewall, which is exactly the setup bypass mode is designed for.

RUN AGENTS IN A DEV CONTAINER

1. **Install Docker Desktop** (docker.com) and **VS Code** with the **Dev Containers** extension.
2. **Add a `.devcontainer/` folder** to your project. Copy Anthropic's reference configuration from the Claude Code repository (`anthropics/claude-code`, folder `.devcontainer`), or ask Claude Code: "set up a dev container for this project based on the official Claude Code devcontainer."
3. **Open the project in the container.**
   Command Palette, then **Dev Containers: Reopen in Container**.
4. **Run your agent inside it** in bypass mode. If it goes wrong, rebuild the container; your host machine is untouched.

> **NOTE**
>
> On Linux and macOS, Claude Code refuses `--dangerously-skip-permissions` when run as **root** or with `sudo`, except inside a recognized sandbox. Dev containers and a normal user account both satisfy this.

2.2 Option 2: a dedicated machine or cloud box

A spare laptop, a small cloud VM, or a cloud dev environment (GitHub Codespaces, a cloud VM, or Claude Code's own cloud sessions at claude.ai/code) all work. Give it its own accounts and tokens, and nothing personal.

2.3 Option 3: a separate user account on your computer

Create a second user account on your Mac or PC just for agents. Your personal files, browser logins, and SSH keys stay out of reach.

2.4 Accounts, tokens, and money

- [ ] A separate GitHub account (or GitHub App) for agents, added as a collaborator only on the repos they work on
- [ ] Fine-grained tokens scoped to those repos, with long-enough expiry
- [ ] A separate Notion teamspace or workspace for agent-writable content
- [ ] Hard monthly spend limits on Anthropic, OpenAI, Google, and cloud accounts
- [ ] Automatic backups (Time Machine, File History, or cloud backups) of anything the agent can write
- [ ] Everything in Git, pushed often, with `main` protected (Chapter 8)

CHAPTER 3

Claude Code (Terminal)

Claude Code has the most complete autonomy controls of any tool. Two settings files matter:

- `~/.claude/settings.json`: **user** settings. Your machine-wide defaults.
- `.claude/settings.json` (project) and `.claude/settings.local.json` (project, just you).

> ⚠ WARNING
>
> `"defaultMode": "auto"` and `"defaultMode": "bypassPermissions"` are **ignored** in project files (`.claude/settings.json` and `.claude/settings.local.json`); this stops a cloned repository from switching your agent to bypass. Put them in `~/.claude/settings.json`, in managed settings, in a `--settings` file, or pass the flag.

3.1 Level 3: auto mode by default

Recent versions of Claude Code already start interactive sessions in **auto mode**, where a classifier model reviews risky actions instead of asking you. To make it explicit and permanent:

~/.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "defaultMode": "auto"
  }
}
```

Then reduce the classifier's false alarms:

- **Teach it your infrastructure.** Run `/auto-mode-setup` to draft `autoMode.environment` entries (trusted repos, buckets, domains, services) from your project and recent sessions, and save them.

- **Approve standing patterns.** Tell it in plain English with specifics ("you may force-push to branches starting with `agent/`"), or add rules under `autoMode.allow`.
- **Check the rules:** `claude auto-mode defaults` prints the built-in block/allow lists; `claude auto-mode config` shows your effective config.

> **NOTE**
> If the classifier blocks 3 actions in a row or 20 in total, auto mode pauses and asks you. These thresholds can't be changed. In `claude -p` runs with no prompt available, the blocked action is skipped and Claude keeps working.

3.2 Level 4: bypass permissions

One session:

TERMINAL

```
claude --dangerously-skip-permissions
```

(Equivalent: `claude --permission-mode bypassPermissions`.) The first time, you accept a warning; Claude Code then records `"skipDangerousModePermissionPrompt": true` in `~/.claude/settings.json` so it doesn't ask again.

Every session, by default:

~/.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "defaultMode": "bypassPermissions"
  },
  "skipDangerousModePermissionPrompt": true
}
```

Start in another mode but keep bypass one keypress away: `claude --allow-dangerously-skip-permissions` adds bypass to the Shift + Tab cycle without starting in it.

A shell alias for convenience:

~/.ZSHRC OR ~/.BASHRC

```
alias yolo='claude --dangerously-skip-permissions'
```

3.3 Fully unattended runs

For scripts, scheduled jobs, and CI, use print mode:

TERMINAL

```
claude -p "Fix the failing tests and commit the result" \
  --dangerously-skip-permissions \
  --permission-prompts none \
  --max-turns 60 \
  --max-budget-usd 10
```

- `--permission-prompts none` denies anything that would still need a human, instead of waiting forever.
- `--max-turns` and `--max-budget-usd` are your seat belts.
- `--output-format json` gives machine-readable results.
- `claude --bg "…"` starts a **background agent** that keeps running after you close the terminal; manage it with `claude agents`. Background sessions honor bypass only if your acceptance is recorded in `~/.claude/settings.json`.
- In CI without bypass, use `--permission-mode dontAsk` with an exact `--allowedTools` list.

3.4 A complete maximum-autonomy settings file

~/.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "defaultMode": "bypassPermissions",
    "allow": [
      "Bash",
      "Edit",
      "Write",
      "WebFetch",
      "WebSearch",
      "mcp__github__*",
      "mcp__notion__*"
    ],
    "additionalDirectories": ["~/projects"]
  },
  "skipDangerousModePermissionPrompt": true,
  "enableAllProjectMcpServers": true,
  "env": {
    "CLAUDE_CODE_DISABLE_DANGEROUS_RM_TIMEOUT": "1"
  }
}
```

What each line buys you:

SETTING	EFFECT
`defaultMode: bypassPermissions`	Every session starts with no prompts
`allow` **rules**	Still useful when you drop into auto or manual mode: these tools stop asking. (Allow rules have no effect *in* bypass, which already allows everything; and broad rules like `Bash` are dropped while in auto mode.)
`mcp__server__*`	Approves every tool from that MCP server. Allow globs must name the server; `mcp__*` alone is ignored
`additionalDirectories`	Lets Claude read and edit outside the folder you started in without asking
`enableAllProjectMcpServers`	Auto-approves servers listed in projects' `.mcp.json` files
`CLAUDE_CODE_DISABLE_DANGEROUS_RM_TIMEOUT`	Removes the 2-minute countdown on critical deletions (the prompt itself stays)

> **TIP**
> Workspace trust: the first time you open a folder, Claude Code asks whether you trust it. Answer once per repository and it remembers. For automation, `-p` runs don't show the dialog, though project allow rules aren't applied there until a folder is trusted.

3.5 Reduce prompts without bypass

If you'd rather stay in Manual or Accept-edits mode for some projects:

- `/fewer-permission-prompts` scans your history and proposes an allowlist.
- `/sandbox`, then **auto-allow**, runs shell commands inside an OS-level sandbox *without prompting* (macOS, Linux, WSL2).
- Answer prompts with **Yes, and don't ask again** to save rules automatically.

3.6 What still prompts in Claude Code, no matter what

STILL ASKS, EVEN IN BYPASS	WHY / WORKAROUND
Tools matching an explicit ask rule	Remove the ask rule
`rm` / `rmdir` **targeting a critical path:** `/`**, top-level folders, your home folder, the working directory or its parents**	A deliberate circuit breaker. In bypass it asks (with a 2-minute countdown that auto-denies). Use literal subfolder paths instead
`AskUserQuestion` **and MCP tools marked requiresUserInteraction**	By design; they need an answer from a person
Connector tools your organization set to "ask"	Admin policy
Reads outside working directories if `blockReadsOutsideWorkingDirectories` **is on**	Leave it off, or add directories
Some cross-session messaging approvals	Only matters with multiple agent sessions messaging each other
`Remove-Item` **on system paths or bare wildcards (PowerShell)**	Denied outright in every mode
Cloud sessions (claude.ai/code)	Bypass isn't offered; **Auto** is the maximum

CHAPTER 4

Claude Desktop, Cowork, and claude.ai

4.1 The Code tab in Claude Desktop

The Code tab uses the same engine and settings files as the CLI.

ENABLE BYPASS IN THE DESKTOP APP

1. **Open Settings** in Claude Desktop and go to **Claude Code**.
2. **Turn on "Allow bypass permissions mode."** On Pro and Max plans this is a personal toggle; on Team and Enterprise, your organization's policy controls it.
3. **Choose the mode.**
   In a session, open the mode selector next to the send button (or press `Cmd` + `Shift` + `M`) and pick **Bypass permissions**, or **Auto** for classifier-reviewed autonomy.
4. **It sticks per folder.**
   The desktop app remembers the mode you pick for each project folder (except Plan). A `defaultMode` in `~/.claude/settings.json` sets the starting point for new folders.

Still asks in the desktop app: the Claude Code list from Chapter 3, plus **safety classifiers on actions Claude takes on external websites** in the built-in browser (any mode), and destructive desktop actions such as deleting a session. On websites, Claude won't purchase things, create accounts, or solve CAPTCHAs without you. Answer a site's first action card with **Always allow** to stop repeat prompts for that site.

4.2 Cowork

Cowork (a tab in Claude Desktop) does longer agentic work on your files, in your browser, and through connectors.

- In the Cowork mode selector, choose **Automatically approve**. On Team and Enterprise, an admin must leave **Allow "Automatically approve" mode** on (it's on by default) in `Organization settings` › `Cowork` › `Permissions`.
- For connector **write** tools to stop asking every task, your admin must also turn on **Allow "Always allow" for connector tools** (off by default on Team and Enterprise). Then choose **Allow for all tasks** in the approval dialog.

4.3 Connectors in claude.ai and the Claude apps

Each connector's tools can be set to **Always allow**, **Needs approval**, or **Blocked**.

MAKE CONNECTORS HANDS-OFF

1. **Open** `Settings` › `Connectors` (or **Customize**, then **Connectors**, in the desktop app).
2. **Click a connector**, then **Tool permissions**.
3. **Set each category or tool to Always allow.**
   Do this for read and write tools you want the agent to use freely.
4. **Repeat for each connector.**

> **NOTE**
> Custom connectors' tools usually lack "read-only" labels, so they default to asking. Set them to **Always allow** explicitly. Organization owners can cap what members may allow, and that cap can't be overridden by individuals.

4.4 Claude in Chrome

The Chrome extension asks per site the first time Claude acts there. Choose **Always allow** for sites you want it to operate on freely. It will still stop for purchases, account creation, and CAPTCHAs, and its safety classifier can still raise a prompt on risky page actions.

CHAPTER 5

Cursor

5.1 The editor's agent

TURN ON RUN EVERYTHING

1. **Open settings:**
   `Settings` › `Agents` › `Approvals & Execution`.
2. **Set the Run Mode to Run Everything.**
   Every tool call, including shell commands, MCP tools, and web fetches, runs automatically, with no sandbox and no classifier.
3. **Read Access** is hidden in Run Everything (nothing is checked). In other modes, set it to **System** to allow reads outside the workspace without asking.

Prefer a safety net? **Auto-review** runs allowlisted calls immediately, sandboxes shell commands where it can, and sends the rest to a classifier. Tune it in plain English in `~/.cursor/permissions.json`:

~/.CURSOR/PERMISSIONS.JSON

```
{
  "autoRun": {
    "allow_instructions": [
      "Any git command in this project, including push to branches starting with agent/.",
      "Installing packages with npm, pnpm, or pip."
    ],
    "block_instructions": []
  }
}
```

The sandbox's network access and extra writable paths live in `~/.cursor/sandbox.json`.

5.2 The Cursor CLI

The command-line agent reads permissions from `~/.cursor/cli-config.json` (global) or `.cursor/cli.json` (project):

~/.CURSOR/CLI-CONFIG.JSON

```
{
  "permissions": {
    "allow": ["Shell(*)", "Read(**)", "Write(**)"],
    "deny": []
  }
}
```

For one-off unattended runs, the CLI's `--force` option runs commands without prompting.

Still asks in Cursor: on a team plan, admin policies and Cursor's built-in security rules sit on top of your local files and can't be weakened by them. In Auto-review or Allowlist modes, commands that need to leave the sandbox ask first, as do writes to protected paths like `.git/config`, `.git/hooks`, `.vscode`, and `.cursorignore`.

CHAPTER 6

VS Code and GitHub Copilot

6.1 Copilot agent in VS Code

LEVEL	HOW
Allow all	Choose **Allow all** in the chat session's permission picker: "runs all tool calls without confirmation"
Autopilot	Choose **Autopilot** in the mode picker: auto-approves tools, retries errors, and answers blocking questions itself
Advanced Autopilot (preview)	`"chat.autopilot.advanced.enabled": true`, where a separate model decides when the task is complete
Every workspace	`"chat.tools.global.autoApprove": true`
This session only	Type `/yolo` (or `/autoApprove`); undo with `/disableYolo`

VS CODE SETTINGS.JSON

```
{
  "chat.tools.global.autoApprove": true,
  "chat.autopilot.advanced.enabled": true
}
```

Finer-grained alternatives: `chat.tools.terminal.autoApprove` (terminal command rules) and `chat.tools.urls.autoApprove` (web addresses). Terminal writes detected *outside* the workspace still ask by default; that's governed by `chat.tools.terminal.blockDetectedFileWrites`.

Running the **Claude agent harness inside VS Code**? Its equivalent of bypass is `"github.copilot.chat.claudeAgent.allowDangerouslySkipPermissions": true`. The **Claude Code extension** has its own **Allow dangerously skip permissions** toggle in its settings, after which **Bypass permissions** appears in its mode indicator.

Still asks in VS Code: tools or terminal rules you've set to `false`; organization (device-management) policies; and, unless you change `blockDetectedFileWrites`, terminal writes it detects outside the workspace. An enabled sandbox still limits file and network access even in

Allow all.

6.2 Copilot CLI

TERMINAL

```
copilot --yolo
```

`--yolo` is an alias for `--allow-all`, which combines `--allow-all-tools`, `--allow-all-paths`, and `--allow-all-urls`. Inside a session, `/yolo` or `/allow-all` does the same; `/reset-allowed-tools` revokes it. Business and Enterprise admins can block these.

6.3 Copilot coding agent (on GitHub)

Assign an issue to Copilot and it works in its own GitHub Actions environment, then opens a pull request. It already runs without prompts. Widen what it can reach in the repository's Settings › Copilot › Coding agent: add a **Custom allowlist** for its firewall, or turn the firewall off entirely (maximum reach, maximum exfiltration risk), and add MCP servers.

CHAPTER 7

Codex, Gemini CLI, and Builders

7.1 OpenAI Codex

GOAL	HOW
Fewer prompts, sandboxed	`codex --full-auto` (workspace-write sandbox, asks only when it needs to leave it)
No prompts, no sandbox	`codex --yolo` (alias of `--dangerously-bypass-approvals-and-sandbox`)
Make it the default	Put the two keys below in `~/.codex/config.toml`

~/.CODEX/CONFIG.TOML

```
approval_policy = "never"
sandbox_mode = "danger-full-access"
```

In the Codex app and IDE extension, choose **Full Access** from the permissions menu.

7.2 Gemini CLI

TERMINAL

```
gemini --yolo
# or
gemini --approval-mode=yolo
```

YOLO mode can only be turned on from the command line, so use an alias (`alias gem='gemini --yolo'`). The settings file `~/.gemini/settings.json` supports `general.defaultApprovalMode` of `"default"`, `"auto_edit"` (auto-approve file edits), or `"plan"`. YOLO also switches on Gemini's sandbox by default, which doesn't add prompts. Leave it on as free protection.

Still asks: if `security.disableYoloMode` or the admin setting `admin.secureModeEnabled` is true, YOLO is refused. Untrusted folders (folder trust) restrict what runs until you trust them.

7.3 Browser app builders

TOOL	MAX-AUTONOMY SETTING
Replit Agent	In the Agent's tools panel, set **Autonomy Level** to **Max** for long, unsupervised runs (labeled beta)
Lovable, Bolt, v0, Base44	They act without per-step approvals already. Give them a connected GitHub repo and backend, and let them run; review via version history

CHAPTER 8

GitHub: Tokens and Repository Settings

GitHub is where agents push code, open pull requests, and trigger automation. Here's how to give them full power with minimal friction.

8.1 Authenticate agents broadly

The GitHub CLI (used by Claude Code, Codex, and others):

TERMINAL

```
gh auth login
gh auth refresh -h github.com -s repo,workflow,read:org,gist,delete_repo
```

The `workflow` scope lets agents edit `.github/workflows/` files; without it, pushes that touch workflows are rejected. `delete_repo` is only needed if you want agents to delete repositories.

A fine-grained personal access token (for MCP servers, n8n, and scripts): go to `Settings` › `Developer settings` › `Personal access tokens` › `Fine-grained tokens`, then **Generate new token**:

OPTION	MAXIMUM-AUTONOMY CHOICE
Resource owner	Your account (or the organization the agent works in)
Expiration	As long as you're comfortable with, or custom
Repository access	**All repositories** (or every repo the agent should touch)
Contents	Read and write
Pull requests	Read and write
Issues	Read and write
Workflows	Read and write
Actions	Read and write
Administration	Read and write (only if agents create repos or change settings)
Metadata	Read (required)

Classic tokens are simpler but blunter: the `repo` and `workflow` scopes cover most agent work.

For teams: install the **Claude GitHub App** (run `/install-github-app` in Claude Code) on **All repositories** so `@claude` works everywhere.

8.2 Repository settings that remove friction

SETTING	WHERE	CHOOSE
Workflow permissions	Settings › Actions › General	**Read and write permissions**
Let Actions open and approve PRs	Same page	Tick **Allow GitHub Actions to create and approve pull requests**
Auto-merge	Settings › General, under Pull Requests	Tick **Allow auto-merge**; agents can then run `gh pr merge --auto --squash`
Branch rules	Settings › Rules › Rulesets	Remove required reviews, or add the agent's account or app to the ruleset's **Bypass list**
Delete merged branches	Settings › General	Tick **Automatically delete head branches**

> **TIP**
>
> A powerful, still-sane setup: leave `main` protected by status checks only (tests must pass), put your agent's account on nothing else, and turn on auto-merge. Agents ship on their own, but only green code lands.

8.3 What still prompts on GitHub

- **Sudo mode:** sensitive actions in the web interface (deleting repos, changing security settings) re-ask for your password or 2FA. Tokens and the API aren't affected.
- **Push protection:** pushes containing detected secrets are blocked until bypassed for that push.
- **Claude's auto mode** blocks merging pull requests no human approved, and force pushes, by default. Use bypass mode, or grant a specific standing approval, if you want agents to do that.
- **Organization policies** (SSO token authorization, required reviews, Copilot policies) override your personal choices.

CHAPTER 9

MCP Servers and Connectors

Every MCP tool call can trigger an approval prompt. Here's how to approve them up front in each host.

HOST	AUTO-APPROVE MCP TOOLS
Claude Code	Allow rules: `"mcp__github__*"`, `"mcp__notion__*"`, one per server (a bare `mcp__*` is ignored). Or bypass mode. `"enableAllProjectMcpServers": true` skips approving `.mcp.json` servers
Claude apps	Connector, then **Tool permissions**, then **Always allow**
Cursor	**Run Everything** covers MCP calls; Auto-review sends them to the classifier
VS Code	**Allow all**, Autopilot, or `chat.tools.global.autoApprove`
Codex	`approval_policy = "never"`
Gemini CLI	`--yolo`, or per server `"trust": true` in its `mcpServers` entry
n8n AI Agent	Don't add human-review steps to its tools

9.1 Give servers enough power

- **Remote OAuth servers** (GitHub, Notion, Linear…) act with *your* account's permissions. Approve every scope they request during sign-in.
- **Token-based servers:** create the token with the broad scopes from Chapter 8 (GitHub) or full capabilities (Notion, Chapter 11).
- **Local servers:** they run with your user's file and network permissions. Point filesystem servers at the folders you want writable.

Still asks: tools a server marks as needing user interaction (`requiresUserInteraction`) always prompt in Claude Code, and so do connector tools your organization set to "ask." ChatGPT's developer-mode connectors may still ask you to confirm write actions.

CHAPTER 10

n8n

10.1 Let AI agents build and run your workflows

OPEN N8N TO AI CLIENTS

1. **Create an API key:**
   `Settings` › `n8n API` › `Create an API key`. Copy it into scripts, Claude Code, or other tools that call n8n's REST API.

2. **Enable instance-level MCP:**
   `Settings` › `Instance-level MCP`, then **Enable MCP access** (owner or admin). Use **Connect** to get the server URL (ending in `/mcp-server/http`) and connect Claude, Cursor, Codex, or ChatGPT with OAuth or an API key.

3. **Expose workflows:**
   under **Workflows enabled**, click **Enable workflows** and add the ones agents may run, or in each workflow's **Settings** toggle **Available in MCP**. Clients can build and edit workflows (n8n 2.13+) as well as run them.

4. **Agents in workflows:**
   in AI Agent nodes, attach the tools you want and *don't* add human-in-the-loop approval to them.

10.2 Self-hosted: unlock everything

TERMINAL

```
# Re-enable nodes n8n 2.0+ blocks by default (Execute Command, etc.)
NODES_EXCLUDE="[]"
```

Set it in your Docker or Docker Compose environment and restart. **Execute Command** runs shell commands on the n8n host. It's powerful and dangerous, so keep self-hosted n8n in a container.

Still asks / limits: n8n Cloud doesn't offer Execute Command at all. The MCP server only lets clients run or see the full contents of workflows you enable for it (`search_workflows` previews everything you can access). Admins can disable MCP or Agents instance-wide.

CHAPTER 11

Notion

11.1 Notion MCP (simplest, full personal access)

Connect `https://mcp.notion.com/mcp` from Claude (Settings › Connectors › Notion), Claude Code, Cursor, or ChatGPT. Signing in with OAuth gives the AI exactly **your** access: every page you can see and edit. Then set its tools to **Always allow** in the host app (Chapter 9).

11.2 An API connection with every capability

For n8n, Zapier, and scripts:

CREATE AN ALL-ACCESS CONNECTION

1. **Open** notion.com/profile/integrations (or Settings › Connections › Develop or manage integrations) and create a new **internal connection** for your workspace. You must be a workspace owner.
2. **Enable every capability:**
   read content, update content, insert content, read comments, insert comments, and read user information including email addresses.
3. **Grant content access broadly.**
   In the connection's **Content access** tab, add every top-level page in each teamspace you want covered. Child pages inherit access.
4. **Copy the API token** into your tools.

11.3 Notion AI agents

Custom Agents run on schedules or triggers. Give each agent the sources and tools it needs in the Agent Builder, and review its run log rather than approving each action. On Business and Enterprise plans, admins control who can create agents, and Custom Agents use paid Notion credits.

Still limited: a connection can never see pages you haven't shared with it. Workspace owners can restrict or revoke MCP clients in `Settings` › `Connections`. Agents run inside Notion's own permission system and can't exceed the access of the content shared with them.

CHAPTER 12

What Always Asks: The Master List

These are the prompts and blocks that survive every setting in this guide. They're worth knowing so you're not surprised when a "fully autonomous" agent pauses.

TOOL	ALWAYS ASKS OR BLOCKS
Claude Code	Critical-path `rm` / `rmdir` (root, home, top-level dirs, working dir); explicit ask rules; `AskUserQuestion`; `requiresUserInteraction` MCP tools; org "ask" connector policies; PowerShell `Remove-Item` on system paths; running bypass as root outside a sandbox
Claude Desktop / Chrome	Purchases, account creation, CAPTCHAs; classifier-flagged actions on external sites; deleting sessions
Cowork	Write connector tools when the admin hasn't allowed "Always allow"
Claude cloud sessions	No bypass; Auto is the ceiling
Cursor	Team admin policies; hard-coded security rules; (outside Run Everything) protected paths and commands leaving the sandbox
VS Code / Copilot	Tools set to `false`; organization policies; enabled sandbox limits
Codex / Gemini CLI	Admin "secure mode" or disabled-YOLO settings; untrusted folders (Gemini)
GitHub	Sudo mode in the web UI; push protection for secrets; org SSO and policies
n8n	Workflows not enabled for MCP; Execute Command unavailable on Cloud
Notion	Pages not shared with a connection; owner-level MCP restrictions
ChatGPT agent	Confirmations before high-impact actions (purchases, sending); "watch mode" on sensitive sites

Your Autonomy Setup Checklist

- [ ] Agents run in a dev container, VM, cloud box, or separate user account
- [ ] Separate GitHub account or app for agents; fine-grained token with full repo permissions
- [ ] Hard spend limits set on every AI and cloud account
- [ ] Everything in Git; `main` protected by status checks; auto-merge allowed
- [ ] `~/.claude/settings.json`: `defaultMode` set to `auto` or `bypassPermissions`, plus `skipDangerousModePermissionPrompt`
- [ ] Claude Desktop: **Allow bypass permissions mode** on; Cowork: **Automatically approve**
- [ ] Every connector tool set to **Always allow**
- [ ] Cursor: **Run Everything** (or Auto-review with allow instructions)
- [ ] VS Code: `chat.tools.global.autoApprove` on, or Autopilot
- [ ] Codex `config.toml`: `approval_policy = "never"`, `sandbox_mode = "danger-full-access"`; Gemini alias with `--yolo`
- [ ] n8n: API key, instance-level MCP on, workflows enabled for MCP; `NODES_EXCLUDE="[]"` if self-hosted
- [ ] Notion: MCP connected; API connection with all capabilities and top-level pages shared
- [ ] Unattended runs have `--max-turns` and `--max-budget-usd` limits

Glossary

Allowlist	A list of pre-approved actions an agent may take without asking.
Auto mode	Claude Code's permission mode in which a classifier reviews actions instead of a person.
Bypass permissions	A mode that skips approval prompts entirely (`--dangerously-skip-permissions` in Claude Code).
Classifier	A second AI model that judges whether an agent's action is safe and matches the request.
Containment	Limiting what an agent can reach (a container, separate accounts, budgets) instead of limiting what it may do.
Critical path	A folder, such as the root, your home folder, or the working directory, that Claude Code protects from deletion in every mode.
Dev container	A Docker container configured as a development environment, used to isolate agents.
Fine-grained token	A GitHub access token limited to chosen repositories and permissions.
Headless / print mode	Running an agent non-interactively from a script, such as `claude -p`.
Prompt injection	Instructions hidden in content an agent reads, meant to hijack its behavior.
Ruleset	GitHub settings that protect branches, with an optional bypass list.
Sandbox	An isolated environment limiting the files and network a command can reach.
Sudo mode	GitHub's re-authentication step for sensitive actions in the web interface.
YOLO mode	Informal name for fully automatic approval in tools like Gemini CLI, Codex, and Copilot CLI.